

Laboratorio I

a.a. 2025/2026

Esercitazione su TypeScript

Roadmap di oggi

In questa lezione implementeremo in TypeScript il primo neurone artificiale: il McCulloch-Pitts' neuron.

In seguito implementeremo in TypeScript una versione semplificata del Rosenblatt's Perceptron: il primo algoritmo di apprendimento per un neurone artificiale.

Come bonus impareremo perchè è difficile fare previsioni metereologiche accurate nel lungo termine :)

Esercizio 1: Classificatore lineare

Scrivere in TypeScript una classe **LinearClassifier** che rappresenti una funzione lineare:

$$s = w_1 \cdot x_1 + \dots + w_n \cdot x_n + \text{bias}$$

Il costruttore:

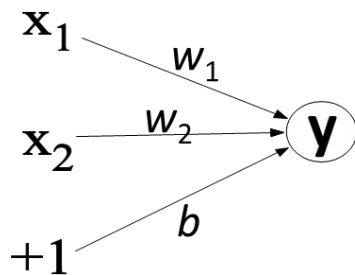
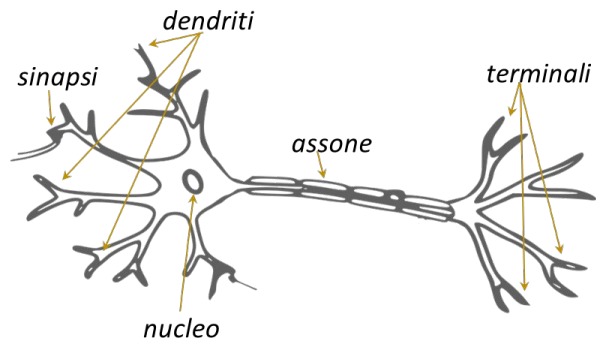
- riceve un array di pesi numerici **weights**=[w1,...,wn],
- riceve un **bias** numerico,
- memorizza **weights** e **bias** come proprietà interne della classe, non accessibili direttamente dall'esterno, ma disponibili a eventuali sottoclassi.

La classe deve supportare due metodi che prendono in input un array numerico x:

- **score(x)**: restituisce il valore di s
- **predict(x)**: restituisce 1 se s>0, 0 altrimenti.

Se la dimensione di x è diversa da quella dei pesi, lanciare un errore.

Il neurone di McCulloch-Pitts



$$\begin{aligned} y &= 1 && \text{se } w_1x_1 + w_2x_2 + b > 0 \\ y &= 0 && \text{se } w_1x_1 + w_2x_2 + b \leq 0 \end{aligned}$$

Input dai neuroni pre-sinaptici: è un vettore $x=[x_1, x_2]$

Pesi sinaptici del neurone: sono le componenti del vettore $[w_1, w_2]$

Bias: un numero b che caratterizza l'inclinazione a "sparare" del neurone.

score(x): input al neurone è la somma di tutti i contributi compreso il bias.

predict(x): è l'output del neurone, 0 è silente, 1 sta "sparando". Rappresenta la decisione binaria del neurone.

Il neurone di McCulloch-Pitts è un classificatore lineare binario!

Esercizio 2: Valutare il neurone su un dataset

Definire un enum **Outcome** che caratterizza i due possibili output del neurone: Silence, Fire.

Un **DataPoint** è una coppia della forma **[sample, label]** dove:

- **sample** è un array omogeneo di tipo generico T.
- **label** è il risultato atteso (0 oppure 1, ovvero Silence oppure Fire)

Per esempio se datapoint è `[[0, 1], 0]` allora significa che il sample a valori di tipo numerico `[0, 1]` deve essere classificato dal neurone come 0.

Definire esplicitamente il tipo di **DataPoint** con l'uso di type.

Un **Dataset** è una collezione (per esempio un array omogeneo) di **DataPoint** usati per addestrare il classificatore.

Considera il seguente dataset:

```
const datasetAND: Dataset = [  
  [[0, 0], 0],  
  [[0, 1], 0],  
  [[1, 0], 0],  
  [[1, 1], 1] // vero se e solo se entrambi veri  
]
```

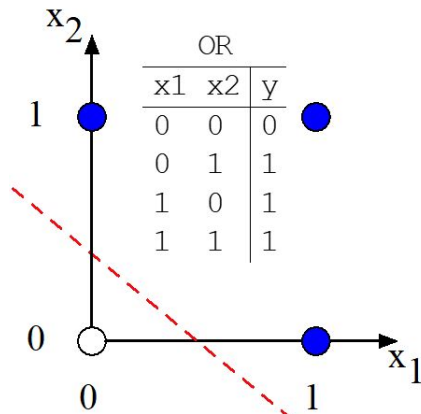
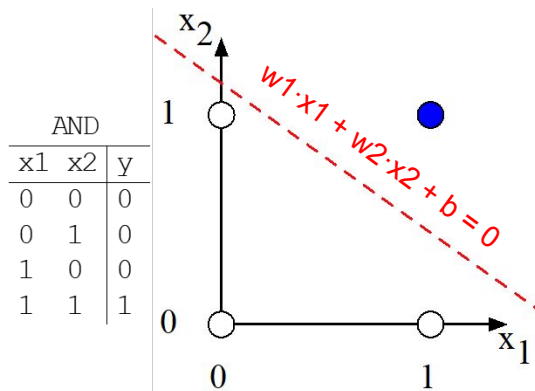
Valutare le predizioni del neurone `new LinearClassifier([1,1],1)` su `datasetAND`. Quanto sono accurate le predizioni del neurone? Trovare dei pesi `[w1,w2]` e bias tali che `new LinearClassifier([w1,w2],bias)` restituisca sempre la predizione corretta per ogni datapoint del `datasetAND`.

Datasets di esempi: tabelle logiche AND e OR

// DATASET DI ESEMPI

```
const datasetAND: Example[] = [  
  [[0, 0], 0],  
  [[0, 1], 0],  
  [[1, 0], 0],  
  [[1, 1], 1] // vero se e solo se entrambi veri  
]
```

```
const datasetOR: Example[] = [  
  [[0, 0], 0], // falso se e solo se entrambi falsi  
  [[0, 1], 1],  
  [[1, 0], 1],  
  [[1, 1], 1]  
]
```



I pesi $[w_1, w_2]$ e il bias b definiscono geometricamente l'iperpiano separatore ai fini della classificazione.

Esiste un metodo automatico per trovare i pesi e il bias giusti per un dato dataset?

Perceptron: il neurone apprende automaticamente

Apprendimento: modifica dei pesi weights e del bias b, in modo da ottenere un certo comportamento desiderato.

Dato un [sample, label], valuto predict() su sample, e calcolo la differenza:
$$\text{errore} = \text{label} - \text{predict}(\text{sample})$$

Se errore è 0, allora non modifico (predizione corretta).

Se errore è 1 (predizione sbagliata), allora modifico weights[i] sommandoci sample[i] e modifico bias incrementandolo di 1.

Intuizione geometrica: muovere l'iperpiano separatore verso la direzione del sample che il neurone ha classificato scorrettamente.

Esempio:

$w_1=0, w_2=0, b=0, [[1,1],1] \rightarrow \text{errore} = 1 \rightarrow \text{Update: } w_1=1, w_2=1, b=1 \rightarrow \text{errore}=0$

Esercizio 3: Perceptron

Si consideri un **dataset**, come definito nell'esercizio 2, cioè una coppia della forma **[sample, label]** dove **sample** è un array numerico, e **label** è il risultato atteso che deve essere di tipo Outcome.

Esempio: $[[0, 1], 0]$ → significa che il sample $[0, 1]$ deve essere classificato dal neurone come 0.

Estendere la classe **LinearClassifier** ad una classe chiamata **Perceptron**, il cui costruttore istanzia una proprietà numerica opzionale **learnRate** con valore di default 1, e aggiungendo due metodi:

- **accuracy(dataset)**: restituisce la frazione di esempi del dataset che sono classificati correttamente dal neurone.
- **train(dataset, epochs)**: modifica **weights** e **bias** per migliorare le predizioni

L'apprendimento è basato su una generalizzazione dell'idea precedente. Per ogni esempio **[sample, label]** del **dataset**:

1. calcolare **predict** su **sample**
2. confrontarla con il valore corretto **label**
3. se la predizione è sbagliata, modificare **weights** e **bias** in base alla seguente regola di aggiornamento.

Regola di aggiornamento del Perceptron:

errore = label - predizione

weights[i] += learnRate * errore * sample[i] // i pesi sinaptici cambiano in base all'errore commesso e dell'input sample

bias += learnRate * errore // similmente per il bias

Ripetere un numero di volte (epochs) questo processo sul dataset. Il parametro epochs è opzionale e settato a 5 di default.

Nota: La proprietà learnRate definisce la velocità di apprendimento, ma valori troppo grandi possono compromettere l'apprendimento.

Si allenì il neurone `new LinearClassifier([1,1],1)` su `datasetAND` e il neurone `new LinearClassifier([0,0],0)` su `datasetOR`.

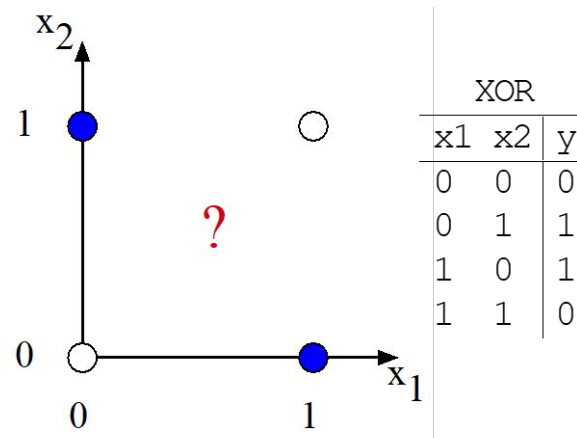
Curiosità e Deep Learning...

La porta logica XOR è impossibile da imparare per il Perceptron.

Questo perchè il Perceptron può apprendere solo funzioni lineari.

Per apprendere XOR ci vuole una rete con almeno due strati di perceptron, dove i neuroni del secondo strato prendono in input l'output dei neuroni del primo strato.

Deep Learning = tanti strati



(Bonus): Mappa Logistica e Caos Deterministico

Scrivere in TypeScript una classe generica **LogisticRNG** basata sulla mappa logistica [Logistic map - Wikipedia](#)

$$x(n+1) = 4 x(n) (1 - x(n))$$

Il costruttore riceve: **seed** iniziale $x(0)$, una funzione **transform** di trasformazione da number a un tipo generico T, una funzione **distance** che calcola la distanza tra due valori di tipo generico T e restituisce un numero. Se il seed non è in $(0,1)$, allora si deve lanciare un errore. La classe deve supportare:

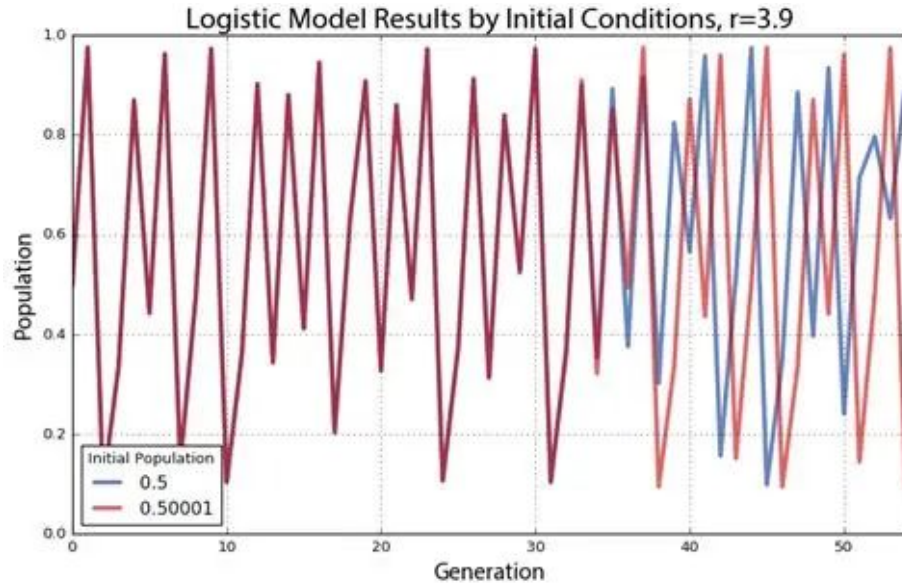
- **values()**: restituisce un generatore dei valori della mappa logistica trasformati tramite transform, partendo da seed
- **divergenceStep(epsilon, steps, threshold)**: restituisce il minimo numero di iterazioni tale per cui la differenza tra la sequenza generata da seed e quella generata da seed + epsilon supera threshold. Se threshold non viene mai superato entro le prime steps iterazioni, allora restituisce -1
- **reduce(steps, f, initialValue)**: esegue una riduzione sui primi steps valori generati dalla mappa logistica, partendo da initialValue e accumulando tramite una funzione generica f
- **reset(seed)**: resetta lo stato iniziale al valore seed. Il parametro seed è opzionale: se non viene fornito, si resetta all'ultimo seed utilizzato.

I valori generati non sono uniformemente distribuiti in $(0,1)$, ma possono essere resi tali tramite la trasformazione

$$u(x) = (2 / \pi) * \arcsin(\sqrt{x})$$

Implementare un metodo statico toUniform(x) che applichi tale trasformazione, e accertarsi calcolando tramite il metodo di classe reduce che la media aritmetica dei valori generati sia circa 0.5

Sensibilità alle condizioni iniziali (butterfly effect)



Q & A

Soluzioni

Esercizi 1, 2 e 3: <https://is.gd/axg2Fj>

Esercizio Bonus: <https://is.gd/ofSw3T>